

Gestión de Proyectos | Aplicación Full Stack

Juan Ignacio de la Torre

Aplicaciones Híbridas

Parcial 2 (Recuperatorio)

Docente: Victor Villafañe

2. Arquitectura de la Aplicación

La aplicación está dividida en dos capas principales: backend y frontend, cada una con responsabilidades claras para mantener la organización y facilitar el mantenimiento.

Backend (Node.js + Express)

El backend se encargó de manejar la lógica, la autenticación y la persistencia de datos. La estructura principal es:

routes/: contiene los endpoints de la API para manejar proyectos y tareas. Cada archivo de rutas organiza las URLs relacionadas con cada entidad.

controllers/: define cómo se procesan las solicitudes y respuestas. Por ejemplo, recibe los datos de un request, llama al service correspondiente y devuelve la respuesta al frontend.

services/: contiene la lógica y operaciones CRUD. Aquí se realizan las validaciones, consultas a la base de datos y manipulación de datos.

middleware/: incluye el middleware de autenticación con JWT. Este verifica que las peticiones a rutas privadas tengan un token válido antes de permitir el acceso.

models/: define las entidades principales del sistema (Usuario, Proyecto, Tarea), sus atributos y relaciones.

Conexión a la base de datos: se utiliza MongoDB para almacenar los datos de usuarios, proyectos y tareas, garantizando persistencia y consultas eficientes.

Frontend (React)

El frontend se desarrolló usando React y está organizado de la siguiente manera:

`components/`: contiene los componentes reutilizables como `ProjectList`, `ProjectForm`, `TaskList` y `TaskForm`, responsables de mostrar la información y manejar la interacción del usuario.

`context/`: se utiliza `AuthContext` para manejar el estado global de la aplicación, especialmente el usuario autenticado y su token JWT.

`pages/`: contiene las vistas principales de la aplicación, como `Login`, `Registro` y `Dashboard`. Cada página integra los componentes necesarios y gestiona la navegación interna.

`React Router DOM`: permite la navegación entre páginas sin recargar la aplicación, manteniendo la experiencia de single-page application (SPA).

`Fetch API`: se usa para comunicarse con el backend, enviando solicitudes autenticadas con el token JWT y consumiendo los datos de la API.

`Hooks y estados`: se implementaron hooks (`useState`, `useEffect`) para manejar los estados locales, refrescar listas de proyectos y tareas, y actualizar la UI dinámicamente según las acciones del usuario.

Flujo resumido de la aplicación

1. El usuario se registra o inicia sesión desde el frontend, proporcionando sus credenciales.
2. El backend valida los datos y, si son correctos, genera un JWT que se envía al frontend.
3. El token se guarda en el Context y se incluye en el header `Authorization` de cada solicitud privada hacia la API.
4. El backend valida el token mediante el middleware antes de permitir acceso a proyectos o tareas, asegurando que cada usuario solo pueda acceder a sus propios datos.
5. El frontend consume la API para realizar operaciones CRUD sobre proyectos y tareas, mostrando en tiempo real la información actualizada al usuario.

Dashboard

Crear Proyecto

Título:

Descripción:

Crear Proyecto

Mis Proyectos

Mi primer proyecto - Controlador de stock

Tareas

Nueva tarea

Agregar

Editar

Eliminar

Hacer el CRUD

Editar

Eliminar

Segundo proyecto - Alquiler de canchas de futbol

Tareas

Nueva tarea

Agregar

Editar

Eliminar

Dashboard principal mostrando proyectos, tareas, formulario de creación de proyectos.

Login

Email

Contraseña

Entrar

Login validado con email y contraseña.

Mis Proyectos

Mi primer proyecto - Controlador de stock

Tareas

Nueva tarea

Agregar

Editar

Eliminar

Hacer el CRUD

Editar

Eliminar

Elegir estilos

Editar

Eliminar

Achicar el código

Editar

Eliminar

Listado de tareas dentro de un proyecto.

Procesos de autenticación y autorización con JWT.

La seguridad de la aplicación se implementó usando JSON Web Tokens (JWT). Al iniciar sesión, el backend genera un token que contiene la información del usuario y lo firma con una clave secreta. Este token se almacena en el frontend mediante Context API y se envía automáticamente en el header Authorization de cada solicitud a rutas privadas.

El backend valida el token con un middleware antes de permitir el acceso, asegurando que solo usuarios autenticados puedan interactuar con sus proyectos y tareas. Por esto cada usuario puede interactuar solo con los datos que le pertenecen, protegiendo su información.